

# Advanced Collections (like Dictionaries), Error Handling, and GitHub

## Dictionary Collections

In the previous module, we stored data into a very rudimentary table by organizing it into a CSV. This week, we are learning about dictionaries which allows us to create tables by using the concept of a “Dictionary”. In these dictionaries, we can consider the columns as “Keys”. These keys can serve as the column names as we make tables while also making them much more useful from a computing standpoint.

Dictionaries are a class in Python and have specific methods much like the Python List class. Some commonly used methods are `items()`, `values()`, and `keys()`.

- `items()`: a way to work with the both the keys and values within a “row” of data in the dictionary

- `values()`: a way to work with only the values of a row (keys not included)

- `keys()`: a method used to work with the keys of a row (values not included)

Using a for loop is one of the commonly used techniques for systematically traversing files, keys, rows, etc. when working with table-like data structures.

## Adding and Removing Data in a Dictionary

When creating a table using a dictionary, rows can be created using the `input()` method to populate the value which is intended for its corresponding key.

For example(from Mod05-Notes):

```

# Get User Input
print("\n--- Adding more data")

id = input("Enter an ID: ")
name = input("Enter an Name: ")
email = input("Enter an Email: ")

row = {"ID": id, "Name": name, "Email": email}

```

---

```

# Add it to the table
table.append(row)

```

In this example, input() is used to collect an ID, Name, and Email. In the script, id, name, and email represent the columns of a table (called “table”). This is aggregated into a row (called “row”) which consists of key-value pairs. The row is added to “table” using the append() method.

To remove a row, one would use the remove() method. Here is an example of how a row can be removed from a dictionary (from Mod05-Notes):

```

# Get User Input
id = input("Enter an ID to remove: ") # Remember input returns a string

# Removed data from the table
for row in table:
    if int(id) == row["ID"]: # Remember to convert to int!
        table.remove(row)

print("\n--- Displaying the existing without the removed data")
for row in table:
    print(row)

```

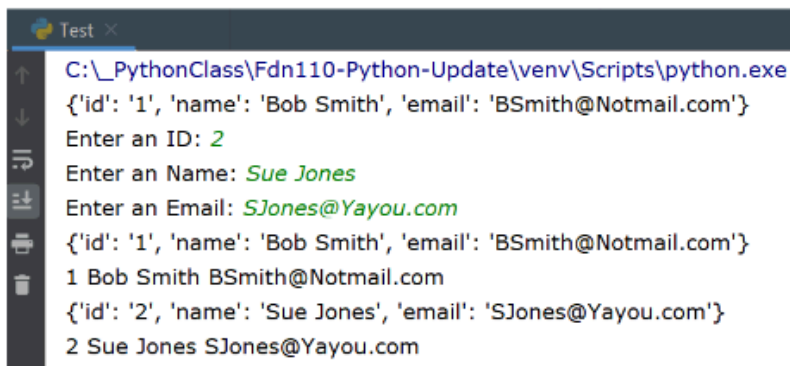
In this example, the user is prompted to enter the id to indicate which row of the table they would like to remove. The user input is converted to a string which is then used by the script as an indicator of which row within the table they would like to remove. The action is completed by

using the remove() method. Finally the table is printed showing the user the data within the table with the row removed.

## Using Dictionaries with File Data

Dictionary data can be used to write data to text files much like lists. One distinction is that unlike lists where CSV was the format, in this week's module the organization of the data is via comma separated key-value pairs. As an illustrated example (from Mod05-Notes):

```
# Write the contents to a file for each row in the table
file = open(FILE_NAME, 'w')
for each_row in table:
    file.write(f'{each_row["ID"]},{each_row["Name"]},{each_row["Email"]}\n')
file.close()
```



```
Test
C:\_PythonClass\Fdn110-Python-Update\venv\Scripts\python.exe
{'id': '1', 'name': 'Bob Smith', 'email': 'BSmith@Notmail.com'}
Enter an ID: 2
Enter an Name: Sue Jones
Enter an Email: SJones@Yayou.com
{'id': '1', 'name': 'Bob Smith', 'email': 'BSmith@Notmail.com'}
1 Bob Smith BSmith@Notmail.com
{'id': '2', 'name': 'Sue Jones', 'email': 'SJones@Yayou.com'}
2 Sue Jones SJones@Yayou.com
```

In this snapshot, a user inputs data which corresponds to key-value pairs. These pairs are separated by commas and encapsulated using curly braces. In this format, the data can be written to a text file. This organizes the data in a systematic way which allows it to be easily processed or accessed by applications.

By using dictionaries, this unlocks certain capabilities as it pertains to reading and writing data to files as it compares to lists. Lists use index values in order to locate corresponding values. On the other hand, dictionaries rely on keys to find a relation for specific values. When organizing data, lists usually store lines as an element. For dictionaries, keys are usually tied to specific values within the line (which makes it possible to work with the data more finely). In general, lists are useful when the data is in a more simple, consistent structure which can be processed sequentially. Dictionaries are useful when data is organized with named attributes.

# JSON Files

JSON (JavaScript Object Notation) is a file type which allows data to bridge the gap between machine readable and user readable. It fits the needs for this module as data is stored with key-value pairs. In this file type, objects are defined with curly braces and arrays are defined with square brackets. Commas separate the key-value elements with arrays as well as the key-value pairs themselves. JSON is a widely-used standard which is helpful in web applications, databases, usage with APIs, and configuration files. JSON is useful typically in more complex use-cases compared to CSV. CSV file format is useful for more simple data types and data structures. JSON is often more human readable and organized, however this often comes at a cost of being more complex and having a larger file-size compared to CSV.

In order to work with JSON files in Python, one will need to import the JSON module in the script. This is typically performed before the bulk of the script starts. The JSON module allows Python to work with JSON files.

## Structured Error Handling (Try-Except)

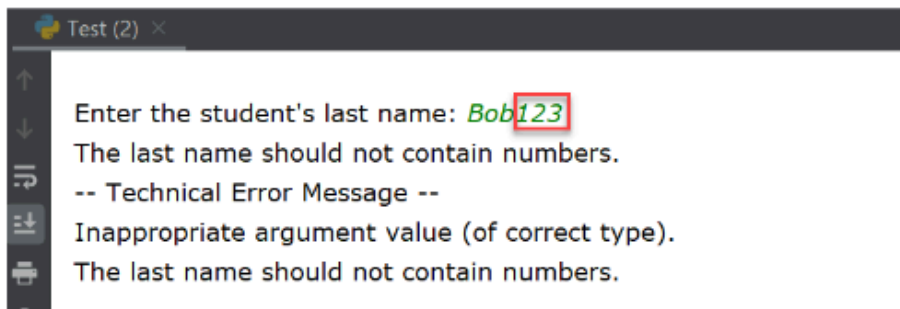
In programming, it is good practice to try to anticipate and get ahead of possible errors which may arise during the course of code execution and application usage. Error handling is one way that can prevent catastrophic failure of code as well as data validation. Error handling can also provide helpful context to users to understand what they might be doing incorrectly in order to remedy unintended errors. Structured error handling can also shield users from technical error messages intended for developers and not necessarily end-users of the application. Instead, developers can provide their own error messaging to users to communicate more relevant information regarding the errors and possible troubleshooting steps.

In error handling, one common construct is the “try-except” block to handle errors. By anticipating error conditions, developers can communicate helpful information to end users. For example (from Mod05-Notes):

```

try:
    # Check that the input does not include numbers
    student_last_name = input("Enter the student's last name: ")
    if not student_last_name.isalpha():
        raise ValueError("The last name should not contain numbers.")
except ValueError as e:
    print(e) # Prints the custom message
    print("-- Technical Error Message -- ")
    print(e.__doc__)
    print(e.__str__())
except Exception as e:
    print("There was a non-specific error!\n")
    print("Built-In Python error info: ")
    print(e, e.__doc__, type(e), sep='\n')

```



```

Test (2) x
↑
↓
Enter the student's last name: Bob123
The last name should not contain numbers.
-- Technical Error Message --
Inappropriate argument value (of correct type).
The last name should not contain numbers.

```

This snapshot illustrates a try-except block which evaluates if user input contains numbers. If so, then the application will throw an error message saying that the user needs to input a value which does not contain numbers.

As another example from Assignment 05, here is an instance where data is validated as alphabetic (throwing an error when a numeric character is detected):

```

---- Course Registration Program ----
  Select from the following menu:
    1. Register a Student for a Course.
    2. Show current data.
    3. Save data to a file.
    4. Exit the program.
-----

What would you like to do: 1
Enter the student's first name: Bob3
The first name must be alphabetic
---Technical Information---
The first name must be alphabetic
Inappropriate argument value (of correct type).
<class 'ValueError'>

---- Course Registration Program ----
  Select from the following menu:
    1. Register a Student for a Course.
    2. Show current data.
    3. Save data to a file.
    4. Exit the program.
-----

What would you like to do: |

```

Note in this case that the program does not stop execution, but rather continues after informing the user of the ValueError and allowing them to make another selection at the menu.

Another type of example of error handling is if a required starter file for Assignment 05 is not detected. Following is the error handling code from my assignment as well as the output as shown in Powershell:

```

# When the program starts, read the JSON file data into the students list(table)
# Extract the data from the file
try:
    file = open(FILE_NAME, "r")
    students=json.load(file)
except FileNotFoundError as e:
    print("File Not Found")
    print("---Technical Information---")
    print(e, e.__doc__, type(e), sep='\n')
    file = open(FILE_NAME, 'w')
except JSONDecodeError as e:
    print("---Technical Information---")
    print(e, e.__doc__, type(e), sep='\n')
    print("Data in file is not valid. Resetting file with current student table data")
    file = open(FILE_NAME, 'w')
    json.dump(students, file)
except Exception as e:
    print("Unhandled exception")
    print("---Technical Information---")
    print(e, e.__doc__, type(e), sep='\n')
finally:
    if not file.closed:
        file.close()

```

```

PS Z:\Python\PythonCourse\_Module05\Assignment> py.exe .\Assignment05.py
File Not Found
---Technical Information---
[Errno 2] No such file or directory: 'Enrollments.json'
File not found.
<class 'FileNotFoundError'>

---- Course Registration Program ----
Select from the following menu:
1. Register a Student for a Course.
2. Show current data.
3. Save data to a file.
4. Exit the program.
-----

What would you like to do:

```

To augment the try-except concept, there is the “finally” block. This makes code easier to read and allows for an organized space to write cleanup code (e.g. closing a file).

# Managing Code Files

Managing code files is important for purposes of collaborating on code, version control, and backups. To store, manage, and collaborate on code, it is often saved in two common places.

One is a network share whereby code/files are stored on a directory which is accessible by users in a common network.

Another is cloud file sharing (for our Module, using GitHub). GitHub is a platform which allows for sharing of code, files, as well as a variety of other functions such as serving as a social network of sorts for developers. GitHub is also extremely useful for tracking work (the base unit of code submissions is a commit). It is useful for serving as an organized portfolio of a GitHub user's work. In GitHub, users can download other users' shared code. GitHub also has powerful version control features. Users can also contribute to other users' projects.